BUSINESS
DEVELOPMENT
GROUP

# Same you
# Just better

**SAME YOU** SINCE 2009
**JUST BETTER**

bdg.am

BUSINESS
DEVELOPMENT
GROUP

# Before we start

# Happy 8th of March 🌷

# Let's talk about your HW
# (Reviewing the Fail2Ban logic)

BUSINESS
DEVELOPMENT
GROUP

# Lesson 6

# Python for DevOps Part 2 OS, JSON/YAML, & Web Frameworks

# Interacting with the Operating System

- The os module lets Python interact with the underlying operating system.
- Crucial for backend developers and DevOps to read configurations without hardcoding them.
- Common uses: navigating directories, creating folders, and reading environment variables.

```python
import os

# Read an environment variable (crucial for secure API keys!)
api_key = os.getenv("MY_SECRET_KEY", "default_value")

# Get the current working directory
current_dir = os.getcwd()
```

## The sys Module

- The sys module provides access to variables and functions that interact with the Python interpreter.
- Perfect for handling command-line arguments when running backend scripts.
- Can also be used to force-exit a script on failure.

```python
import sys

# Print the arguments passed to the script
print("Script name:", sys.argv[0])

if len(sys.argv) < 2:
    print("Error: Missing argument!")
    sys.exit(1) # Exit with an error code
```

# File Handling

- While os handles paths, shutil handles the actual files.Used for copying, moving, and archiving files/directories.
- To open a file, we can use open() function, which requires file-path and mode as arguments.

```python
f = open("geek.txt", "r")
print("Filename:", f.name)
print("Mode:", f.mode)
print("Is Closed?", f.closed)

f.close()
print("Is Closed?", f.closed)
```

```python
import shutil

# Copy a configuration file
shutil.copy("app_config.yml", "app_config_backup.yml")

# Archive a directory
shutil.make_archive("logs_backup", "zip", "/var/logs/app")
```

# Why Do We Need JSON & YAML?

- Backend services rarely talk in plain text; they use structured data.
- **JSON (JavaScript Object Notation)**: The language of the web and APIs. Strict syntax (curly braces, quotes).
- **YAML (YAML Ain't Markup Language)**: The language of configuration (Docker, Kubernetes). Relies on indentation.
- Both translate perfectly into Python Dictionaries!

## JSON

```
{
"employees":
   [
     {
      "id": "40159",
      "name": "John",
      "technology": "Cloud computing",
      "title": "Engineer",
      "team": "Development"
     }
   [
}
```

## YAML

```
employees:
   - id: '40159'
     name: John
     technology: Cloud computing
     title: Engineer
     team: Development
```

# When to use YAML vs. JSON

- JSON is often preferred over YAML due to its widespread support and integration with JavaScript, making it popular for distributed software, web apps, configuration, and APIs.
- Though YAML is more human-readable and supports data typing, JSON is favored for cross-compatibility as most applications already parse it.
- However, YAML is prevalent in DevOps and IaC, notably for configuration in tools like Docker and Kubernetes, due to its readability and support for comments.

# Data Structures: JSON

- Python has a built-in json module.
- json.loads(): Converts a JSON string into a Python dictionary.
- json.dumps(): Converts a Python dictionary back into a JSON string.

```python
import json

# some JSON:
x =  '{ "name":"John", "age":30, "city":"New York"}'

# parse x:
y = json.loads(x)

# the result is a Python dictionary:
print(y["age"])
```

```python
import json

# a Python object (dict):
x = {
  "name": "John",
  "age": 30,
  "city": "New York"
}

# convert into JSON:
y = json.dumps(x)

# the result is a JSON string:
print(y)
```

# Reading & Writing JSON Files

- When working with files, we drop the "s" (string) and use load() and dump().

```python
import json

# Read a JSON file
with open('data.json', 'r') as file:
    data = json.load(file)

# Write to a JSON file (with pretty formatting)
with open('output.json', 'w') as file:
    json.dump(data, file, indent=4)
```

## Parsing YAML in Python

- Unlike JSON, YAML requires an external library: PyYAML.
- Always use yaml.safe_load() instead of yaml.load() for security (prevents arbitrary code execution).

**geeksforgeeks.yml**

```
UserName: GeeksforGeeks
Password: GFG@123
Phone: 1234567890
Website: geeksforgeeks.org
Skills:
  -Python
  -SQL
  -Django
  -JavaScript
```

```python
import yaml

# YAML file path
yaml_file = 'geeksforgeeks.yml'

# Reading YAML data from file
with open(yaml_file, 'r') as f:
    yaml_data = yaml.load(f, Loader=yaml.FullLoader)

# Displaying parsed YAML data
print(yaml_data)

"""
output:
{'UserName': 'GeeksforGeeks', 'Password': 'GFG@123', 'Phone': 1234567890,
'Website': 'geeksforgeeks.org', 'Skills': '-Python -SQL -Django -Javascript'}
"""
```

# The Backend Web Frameworks

- Backend developers use frameworks to build the APIs and web apps that DevOps engineers deploy.
- A framework handles routing (URLs), HTTP requests, and responses so you don't have to write them from scratch.
- Choosing the right framework dictates how complex deployment and scaling will be.

## Django: The Heavyweight

- Philosophy: "Batteries included."
- Comes with an ORM (Database connector), admin panel, and authentication out of the box.
- Pros: Extremely fast to build complex, full-featured web applications.
- Cons: Heavy, steep learning curve, monolithic structure makes microservices harder.

Flask: The Microframework

- Philosophy: "Do one thing well."
- Provides just the essentials (routing and request handling). You add plugins for databases or auth.
- Pros: Lightweight, incredibly easy to learn, perfect for small APIs and webhooks.
- Cons: Scaling up requires managing many third-party extensions.

FastAPI: The Modern Standard

- Philosophy: Speed and developer experience.
- Built on modern Python features (type hints) and is asynchronous by default.
- Pros: Blazing fast, auto-generates interactive API documentation (Swagger UI).
- Cons: Newer ecosystem compared to Django/Flask.

FastAPI

python

# Frameworks: Comparison

| Feature | Django | Flask | FastAPI |
|---|---|---|---|
| Size | Monolithic / Heavy | Micro / Light | Micro / Light |
| Speed | Moderate | Fast | Extremely Fast |
| Best for | Full web apps (CMS, E-commerce) | Simple APIs, Webhooks | High-performance microservices |
| Learning Curve | Steep | Easy | Moderate |

Python frameworks cannot directly talk to the internet securely or efficiently.
They require an interface server:

- WSGI (Web Server Gateway Interface): Used by Django/Flask (e.g., Gunicorn).
- ASGI (Asynchronous Server Gateway Interface): Used by FastAPI (e.g., Uvicorn).

As a DevOps engineer, you will containerize the app and its WSGI/ASGI server.

# WSGI vs ASGI

| FEATURE | DEFAULT INTERFACE | ASGI SUPPORT |
|---|---|---|
| Request Handling | Synchronous only | Sync + Async |
| WebSocket Support | Not available | Fully supported |
| Streaming / Push | Limited | Native |
| Performance | Slower under high concurrency | Optimized for I/O & concurrency |
| Real-Time Features | Hacky / workaround needed | Out-of-the-box |
| Complexity | Easier to reason about | Requires async knowledge |

## Fetching Data with requests

- Backend apps often need to talk to other apps (Public APIs, Databases, microservices).
- The requests library is the standard for making HTTP calls in Python.
- We can fetch external JSON data and analyze it within our framework.

```python
import requests

x = requests.get('https://w3schools.com/python/demopage.htm')

print(x.text)
```

# Making a GET Request

- Send a request to a URL and capture the response.
- Check the status code (200 OK, 404 Not Found, 500 Server Error).

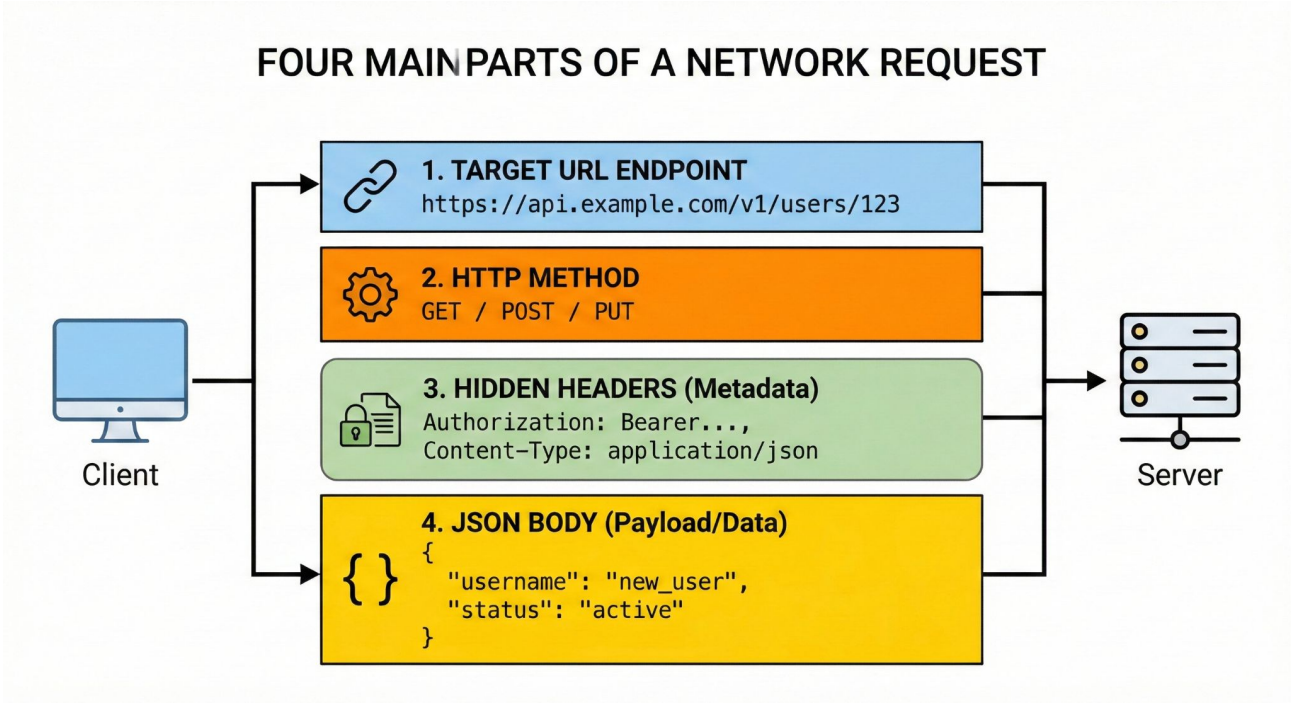
```python
import requests

response = requests.get("https://api.github.com/users/octocat")

if response.status_code == 200:
    data = response.json() # Automatically converts JSON to a Dictionary!
    print(data["public_repos"])
```

# The Anatomy of a Backend Endpoint

- Receive: Client sends a request to a route (e.g., /analyze).
- Process: Python code executes (fetches DB, calculates metrics, reads files).
- Respond: Framework packages the dictionary back into JSON and returns it.



# The Golden Rule of Backend Secrets

- NEVER hardcode API keys, passwords, or IP addresses in your Python code.
- Use the os module to read them from the environment.
- This makes your code portable for Docker and Kubernetes.

```python
import os

# The code doesn't know the password, the server does!
DB_PASSWORD = os.getenv("DB_PASS")
```

```python
from dotenv import load_dotenv
import os

# Load variables from .env file
load_dotenv()

# Access the variables
database_url = os.getenv("DATABASE_URL")
secret_key = os.getenv("SECRET_KEY")

print(f"Database URL: {database_url}")

```

- Homework 6: Project Genesis (The Seed App)
- The Mission: Build the foundational Python backend for your long-term project! Pick a theme you love (Finance, Weather, Gaming). Hint: Next lesson, we add a Database!
- Next Lesson: Python for DevOps Part 3.

# Questions?

BUSINESS
DEVELOPMENT
GROUP

# CONTACT US

info@bdg.am | +374 55 201209 | 4 Hr. Kochar Street, Yerevan